

EnterprateAI Live Business Plan

Technical Design & Implementation Specification

Product: EnterprateAI

Module: Live Business Plan

Version: 1.0

Audience: Engineering / Development Team

Dependency: Existing EnterprateAI Ontology, Intelligence Engines, Scenario Engine, Credit System and Third-Party Integration Layer

Target Subscription: Decision Engine and above

Priority: Critical

1. Purpose

This document defines how the Live Business Plan described in the Product Requirements Document should be implemented technically within the existing EnterprateAI architecture.

The development team must build a system that connects:

1. Business Plan assumptions
2. Business Ontology
3. Actual operational data
4. Existing third-party integrations
5. KPI monitoring
6. Viability Engine
7. Survival Engine
8. Stability Engine
9. Growth Engine
10. Master/Fragility Intelligence
11. Scenario Simulation
12. Decision Adoption
13. Live Plan versioning
14. Decision outcome monitoring
15. AI narrative generation

The feature must preserve EnterprateAI's deterministic-first architecture.

AI must interpret and communicate calculated intelligence.

AI must not become the authoritative source of business metrics.

2. Existing Components to Reuse

The engineering team must NOT rebuild capabilities that already exist.

2.1 Existing Business Ontology

Reuse existing ontology entities including:

- Business
- BusinessProfile
- Offering
- PricingModel
- RevenueStream
- Customer
- CustomerSegment
- Contract
- Opportunity
- DemandEvidence
- CashAccount
- Transaction
- RevenueRecord
- ExpenseRecord
- CashflowSnapshot
- Staff
- Role
- Process
- Dependency
- CapacityProfile
- CapacitySnapshot
- Backlog
- MarketProfile
- Channel
- Competitor
- PositioningProfile
- GrowthSignal
- Assessment
- AssessmentMetric
- AssessmentScore

- RiskFlag
- Recommendation
- Scenario
- ScenarioChange
- SimulationRun
- SimulationResult

Do not introduce duplicate versions of these entities inside Live Business Plan.

3. Existing Integrations

EnterpriseAI already has integration capability for external systems including QuickBooks, Zoho and other supported providers.

Therefore:

The Live Business Plan module must NOT rebuild:

- OAuth flows
- API credential management
- provider connection screens
- provider-specific authentication
- existing synchronisation mechanisms

The Live Business Plan module must consume data made available through the existing EnterpriseAI Integration Layer.

4. Integration Architecture

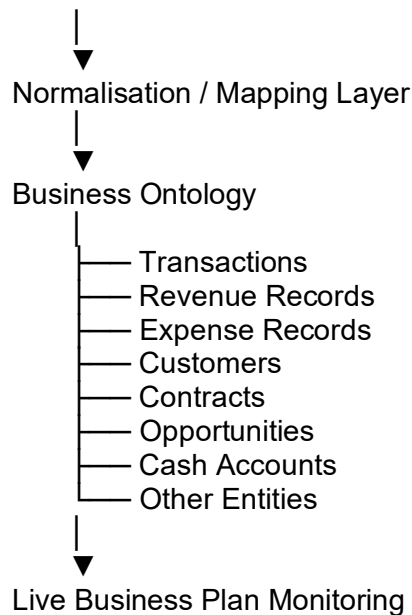
Existing connectors should feed normalised information into the Business Ontology.

Conceptually:

QuickBooks
Xero
Zoho
Stripe
PayPal
Other Sources



Existing EnterpriseAI Integration Layer



The Live Business Plan does not need to know whether revenue originated from QuickBooks, Xero, Zoho or EnterpriseAI itself.

It should primarily read normalised ontology entities.

5. Core Architectural Principle

The system must maintain three logically distinct states.

ACTUAL STATE

What is actually happening in the business

PLAN STATE

What management currently intends to happen

SCENARIO STATE

What might happen if a decision is made

These must never be combined incorrectly.

6. State Architecture

6.1 Actual State

Actual State comes from the Business Ontology.

Examples:

RevenueRecord
ExpenseRecord
Transaction
Customer
Contract
Staff
CashAccount
CapacitySnapshot

Actual State represents confirmed historical/current reality.

Historical actual records must not be rewritten by Live Business Plan operations.

7. Plan State

Plan State must NOT be created by altering Actual State.

Instead create a:

Planned-State Overlay

This layer represents future assumptions.

Example:

Actual:

Staff = 4

Live Plan:

Staff = 4 until 30 September

Staff = 5 from 1 October

Do NOT create the fifth employee in the actual staff table simply because the hiring decision was adopted.

Instead store:

entity_type: Staff
operation: ADD
effective_from: 2026-10-01
status: PLANNED

When the employee actually joins, the corresponding actual Staff record may subsequently appear.

The system can then determine whether the planned action occurred.

8. Scenario State

Scenario State already exists through:

Scenario
ScenarioChange
SimulationRun
SimulationResult

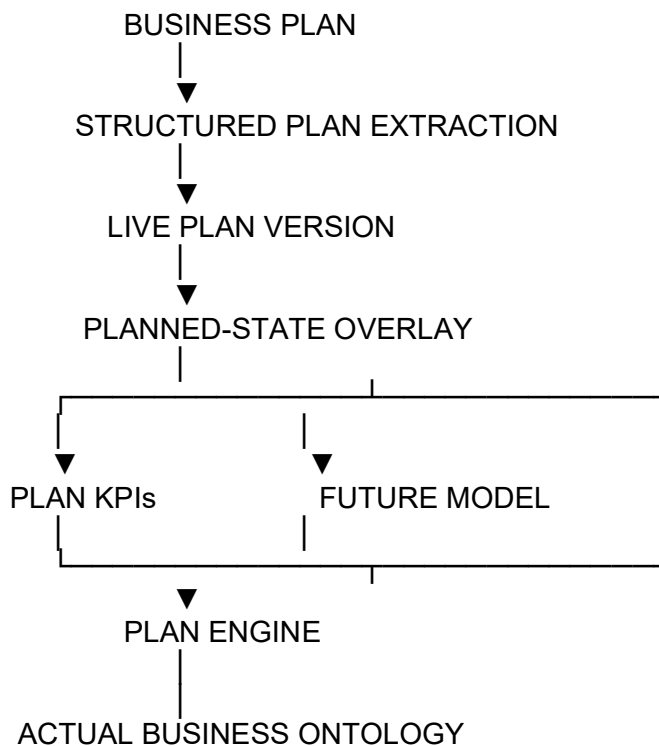
Scenario changes remain hypothetical.

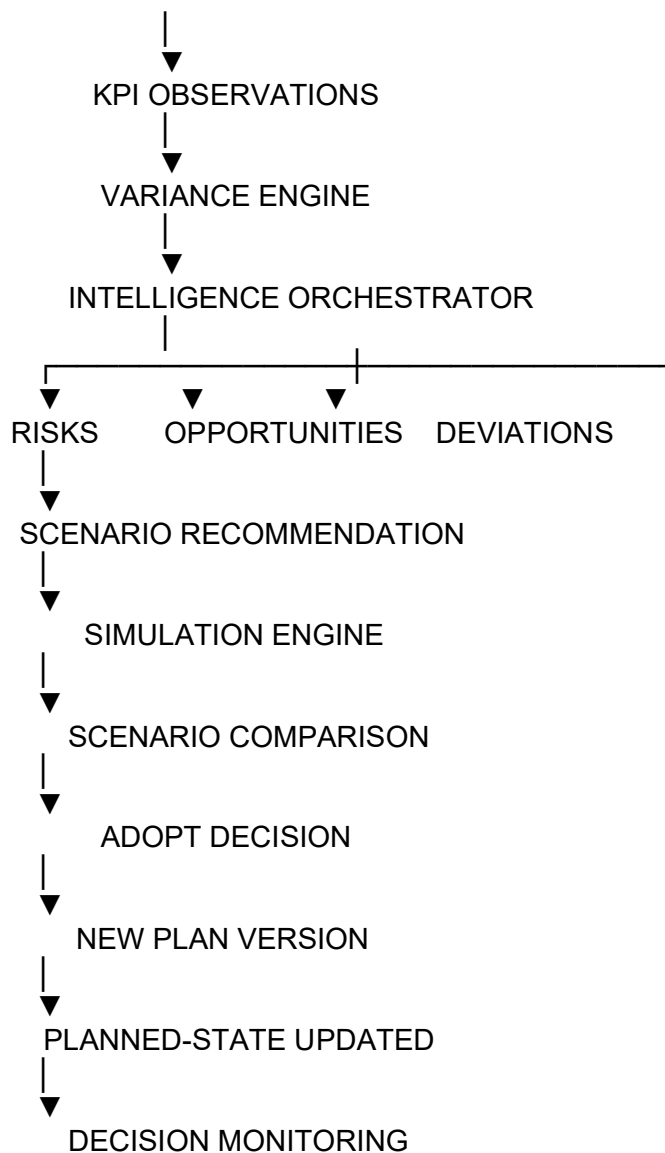
A scenario must never mutate:

- actual ontology data
- Live Plan assumptions
- accounting records

until explicitly adopted.

9. Overall Architecture





10. New Domain Objects

The following entities should be added.

11. live_business_plans

Represents the currently active Live Business Plan for one business.



live_business_plans

id UUID PK
business_id UUID FK
status ENUM
current_version_id UUID
source_document_id UUID nullable
adopted_at TIMESTAMP
created_by UUID
created_at TIMESTAMP
updated_at TIMESTAMP

Statuses:

DRAFT
ACTIVE
SUSPENDED
ARCHIVED

One business should normally have only one ACTIVE Live Business Plan.

12. live_plan_versions

Every plan change creates an immutable version.

live_plan_versions

id UUID PK
live_plan_id UUID FK
version_number VARCHAR
previous_version_id UUID nullable
source_type ENUM
source_reference_id UUID nullable
change_summary TEXT
approved_by UUID
approved_at TIMESTAMP
created_at TIMESTAMP

Source types:

INITIAL_ADOPTION
MANUAL_UPDATE
SCENARIO_ADOPTION
ANNUAL_REVIEW
IMPORTED_PLAN
GENERATED_PLAN
ADMIN_CORRECTION



Examples:

- 1.0 Initial Plan
- 1.1 Pricing Decision
- 1.2 Hiring Decision
- 1.3 Marketing Expansion

Versions are immutable after approval.

13. live_plan_assumptions

Stores structured plan values.

live_plan_assumptions

id UUID PK
plan_version_id UUID FK
business_id UUID FK

domain ENUM
entity_type VARCHAR
entity_id UUID nullable

metric_code VARCHAR
assumption_name VARCHAR

value_numeric DECIMAL nullable
value_text TEXT nullable
value_boolean BOOLEAN nullable

unit VARCHAR nullable
currency VARCHAR nullable

period_start DATE nullable
period_end DATE nullable

effective_from DATE nullable
effective_to DATE nullable

source_type VARCHAR
source_reference_id UUID nullable

confidence_level ENUM

created_at TIMESTAMP



Domains:

COMMERCIAL
FINANCIAL
CUSTOMER
OPERATIONAL
GROWTH
MARKET
STRATEGIC

14. planned_entity_changes

Used where a planned decision modifies an ontology entity.

Example:

Hiring one employee.

planned_entity_changes

id UUID PK
plan_version_id UUID FK
business_id UUID FK

entity_type VARCHAR
entity_id UUID nullable

operation ENUM

field_name VARCHAR nullable
old_value JSONB nullable
planned_value JSONB

effective_from DATE
effective_to DATE nullable

source_decision_id UUID nullable

status ENUM

created_at TIMESTAMP

Operations:

CREATE
UPDATE
REMOVE

Statuses:

PLANNED
DUE
COMPLETED
MISSED
CANCELLED
SUPERSEDED

This table is extremely important.

It prevents future decisions from contaminating actual ontology state.

15. plan_kpis

Defines the KPIs being monitored.

plan_kpis

id UUID PK
live_plan_id UUID FK
business_id UUID FK

metric_code VARCHAR
metric_name VARCHAR
domain VARCHAR

aggregation_type VARCHAR
measurement_frequency ENUM

target_type ENUM
target_value DECIMAL nullable
target_formula JSONB nullable

tolerance_warning_pct DECIMAL
tolerance_critical_pct DECIMAL

source_entity VARCHAR
source_metric VARCHAR

higher_is_better BOOLEAN nullable

active_flag BOOLEAN

created_at TIMESTAMP
updated_at TIMESTAMP

Frequencies:

DAILY
WEEKLY
MONTHLY
QUARTERLY
ANNUAL
EVENT_DRIVEN

16. Example KPI Codes

Initial standard library:

revenue_monthly
revenue_annual
gross_margin_pct
net_margin_pct
fixed_cost_monthly
variable_cost_monthly
cash_balance
cash_runway_months
burn_rate
net_cashflow
customer_count
new_customers
customer_concentration_top1
customer_concentration_top3
recurring_revenue_pct
capacity_utilisation
backlog_volume
employee_count
sales_pipeline_value
conversion_rate
break_even_months

17. kpi_observations

Stores calculated actual observations.

This is NOT the authoritative source of actual business data.

It is a materialised measurement generated from ontology data.

kpi_observations



id UUID PK
business_id UUID FK
live_plan_id UUID FK
kpi_id UUID FK

period_start DATE
period_end DATE

actual_value DECIMAL

source_type VARCHAR
source_entities JSONB

confidence_score DECIMAL
confidence_level ENUM

calculated_at TIMESTAMP

18. plan_variances

plan_variances

id UUID PK
live_plan_id UUID FK
kpi_id UUID FK

plan_version_id UUID FK

period_start DATE
period_end DATE

planned_value DECIMAL
actual_value DECIMAL

variance_absolute DECIMAL
variance_percentage DECIMAL nullable

severity ENUM
direction ENUM

detected_at TIMESTAMP
resolved_at TIMESTAMP nullable

status ENUM



Severity:

INFO
MONITOR
WATCH
HIGH
CRITICAL

Direction:

ABOVE_PLAN
BELOW_PLAN
ON_PLAN

19. adopted_decisions

adopted_decisions

id UUID PK
business_id UUID FK
live_plan_id UUID FK

scenario_id UUID FK
simulation_run_id UUID FK

plan_version_before UUID FK
plan_version_after UUID FK

decision_name VARCHAR
decision_type VARCHAR

decision_summary TEXT
rationale TEXT nullable

effective_from DATE
monitoring_start_date DATE
monitoring_end_date DATE nullable

adopted_by UUID
adopted_at TIMESTAMP

status ENUM

Statuses:

PLANNED
ACTIVE

ACHIEVED
PARTIALLY_ACHIEVED
UNDERPERFORMING
FAILED
CANCELLED
SUPERSEDED

20. decision_expected_outcomes

Each adopted decision should store the expectations produced during simulation.

decision_expected_outcomes

id UUID PK
adopted_decision_id UUID FK

metric_code VARCHAR

baseline_value DECIMAL nullable
expected_value DECIMAL
expected_delta DECIMAL nullable

target_date DATE
tolerance_pct DECIMAL nullable

engine_name VARCHAR nullable

created_at TIMESTAMP

21. decision_outcome_observations

decision_outcome_observations

id UUID PK
adopted_decision_id UUID FK
expected_outcome_id UUID FK

measurement_date DATE

expected_value DECIMAL
actual_value DECIMAL

variance DECIMAL



variance_pct DECIMAL

status ENUM

created_at TIMESTAMP

Statuses:

ON_TRACK

WATCH

UNDERPERFORMING

ACHIEVED

FAILED

22. live_plan_alerts

live_plan_alerts

id UUID PK

business_id UUID FK

live_plan_id UUID FK

alert_type VARCHAR

source_type VARCHAR

source_reference_id UUID nullable

severity ENUM

headline VARCHAR

deterministic_explanation TEXT

recommended_scenario_type VARCHAR nullable

status ENUM

created_at TIMESTAMP

acknowledged_at TIMESTAMP nullable

resolved_at TIMESTAMP nullable

Statuses:

OPEN

ACKNOWLEDGED

RESOLVED

DISMISSED

23. narrative_sections

Store the readable Business Plan separately from structured assumptions.

live_plan_narrative_sections

id UUID PK

plan_version_id UUID FK

section_code VARCHAR

section_title VARCHAR

section_order INTEGER

content TEXT

generated_by ENUM

source_references JSONB nullable

created_at TIMESTAMP

Generated by:

AI

USER

IMPORTED

SYSTEM

Narrative must never be the source of KPI calculations.

24. Plan Adoption Workflow

Step 1

User selects:

Adopt as Live Business Plan

Step 2

Determine source:

GENERATED_PLAN
IMPORTED_PLAN
EXISTING_BUSINESS_DATA

Step 3

Extract structured assumptions.

Step 4

Map assumptions to ontology concepts.

Example:

"Expected monthly sales: £20,000"

becomes:

```
{  
  "domain": "FINANCIAL",  
  "metric_code": "revenue_monthly",  
  "value_numeric": 20000,  
  "currency": "GBP"  
}
```

25. Extraction Rules

AI may assist with extracting assumptions from narrative Business Plans.

However:

AI extraction result must be treated as:

PROPOSED

until confirmed.

Workflow:

Document



AI extraction



Structured candidate assumptions





Validation



User confirmation



Live Plan

Never:

Document



AI



Automatically write authoritative values

26. Plan Validation

Before adoption, validate:

- financial assumptions
- missing dates
- duplicate assumptions
- contradictory values
- unsupported metrics
- invalid currencies
- invalid percentages
- negative values where prohibited

The system should surface validation warnings to the user.

27. Baseline Assessment

Immediately after Plan adoption:

Run the existing intelligence stack.

Viability

Survival

Stability

Growth

Master Intelligence / Fragility

Store the Assessment as the baseline associated with Live Plan Version 1.0.

28. KPI Mapping Engine

Implement a service:

LivePlanKPIService

Responsibilities:

1. Load active KPIs.
2. Retrieve planned values.
3. retrieve actual ontology values.
4. calculate observations.
5. calculate variance.
6. classify severity.
7. persist results.
8. trigger intelligence recalculation if thresholds require it.

29. KPI Calculation Registry

Do not hardcode formulas inside controllers.

Create configurable KPI definitions.

Example:

```
{
  "metric_code": "gross_margin_pct",
  "calculation_type": "FORMULA",
  "formula": "(revenue-variable_cost)/revenue",
  "required_sources": [
    "RevenueRecord",
    "ExpenseRecord"
  ]
}
```

Example:

```
{
  "metric_code": "customer_concentration_top1",
  "calculation_type": "ENGINE",
  "engine": "stability",
  "metric": "top_customer_pct"
}
```

30. Data Refresh Behaviour

Live Business Plan should react whenever relevant ontology data changes.

Sources may include:

- native EnterpriseAI actions
- completed integration syncs
- imported data
- manual entries

Preferred event:

BUSINESS_DATA_UPDATED

Payload example:

```
{
  "business_id": "uuid",
  "source": "quickbooks",
  "entity_types": [
    "Transaction",
    "RevenueRecord",
    "ExpenseRecord"
  ],
  "updated_at": "timestamp"
}
```

The Live Business Plan module then determines which KPIs need recalculation.

31. Integration Sync Ownership

The Live Business Plan module must NOT control provider sync schedules.

Existing integration services remain responsible for:

- when to sync
- how to sync
- authentication
- provider API error handling

The Live Plan module starts processing once normalised data is available.

32. Incremental Recalculation

Do not recalculate every KPI and every engine after every transaction.

Implement dependency mapping.

Example:

ExpenseRecord changed

Affected:

monthly_expenses
net_cashflow
burn_rate
runway
profitability
Survival
Viability
Master Score

Potentially unaffected:

market_position
brand_strength

This reduces unnecessary computation.

33. Dependency Registry

Recommended structure:

metric_dependencies

source_entity
metric_code
engine_name

Example:

RevenueRecord
→ revenue_monthly
→ gross_margin
→ customer_concentration
→ Survival
→ Stability
→ Viability

34. Variance Engine

Implement:

VarianceEngine

Input:

planned value
actual value
tolerance
metric semantics

Output:

variance
variance percentage
severity
direction

Example:

Plan Revenue = £25,000
Actual Revenue = £18,000

Variance = -£7,000
Variance % = -28%
Severity = CRITICAL

35. Variance Rules

Initial defaults:

0–5% ON_TRACK
5–10% MONITOR
10–20% WATCH
20–30% HIGH
>30% CRITICAL

These must be configurable.

Some metrics require inverse interpretation.

Example:

Revenue:

Higher generally positive.

Costs:

Higher than plan generally negative.

Customer concentration:

Higher is negative.

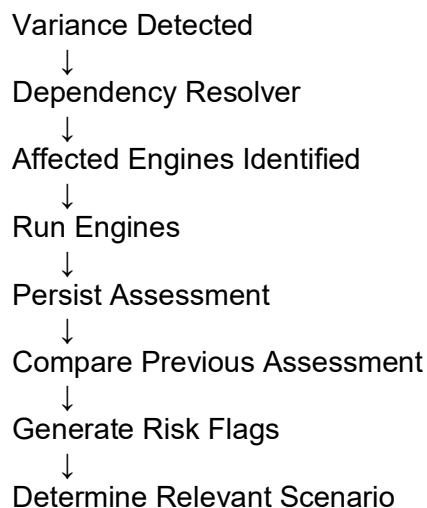
Capacity utilisation:

Too low and too high may both be undesirable.

Therefore use metric-specific variance logic where necessary.

36. Intelligence Orchestration

When material variance occurs:



37. Scenario Recommendation Engine

Scenario recommendations should initially be deterministic.

Examples:

LOW_RUNWAY_RISK
→ Cash Recovery Scenario
→ Cost Reduction Scenario
→ Funding Scenario

CUSTOMER_CONCENTRATION_RISK

- Major Customer Loss Scenario
- Customer Diversification Scenario

CAPACITY_CONSTRAINT

- Hiring Scenario
- Contractor Scenario
- Automation Scenario
- Price Increase Scenario

LOW_MARGIN_RISK

- Pricing Scenario
- Supplier Cost Reduction Scenario

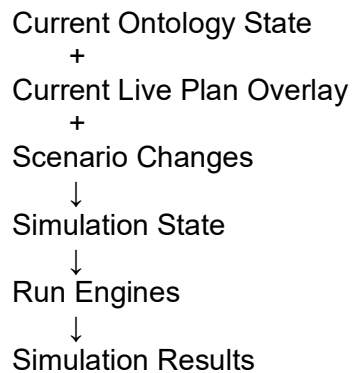
AI may explain WHY the scenario is relevant.

AI should not be necessary to determine that the scenario exists.

38. Scenario Execution

Reuse the existing Scenario architecture.

Sequence:



The simulation state should exist:

- in memory
- temporary calculation storage

It must NOT alter Actual or Plan State.

39. Simulation Context

Simulation execution must know which baseline is being evaluated.

Recommended:

CURRENT_ACTUAL
CURRENT_PLAN
FUTURE_PERIOD

Example:

Hiring in October should simulate against the expected October plan state rather than only today's business state.

40. Simulation Result

Simulation output should include:

```
{
  "base": {
    "viability_score": 72,
    "survival_score": 68,
    "stability_score": 58,
    "growth_score": 64
  },
  "simulated": {
    "viability_score": 70,
    "survival_score": 59,
    "stability_score": 73,
    "growth_score": 79
  },
  "delta": {
    "viability": -2,
    "survival": -9,
    "stability": 15,
    "growth": 15
  }
}
```

Also return:

- new risks
- removed risks
- unchanged risks
- breakpoint information where available

- affected KPIs

41. Adopt Decision Service

Implement:

DecisionAdoptionService

This must be a transactional operation.

Input:

business_id
scenario_id
simulation_run_id
effective_from
user_id

42. Adoption Validation

Before adoption validate:

- simulation belongs to business
- simulation completed successfully
- simulation is based on current or valid baseline
- user has permission
- Live Plan is ACTIVE
- effective date is valid
- scenario has not already been adopted
- conflicting planned changes do not exist

43. Adoption Transaction

Within a single database transaction:

1. Lock active Live Plan.
2. Retrieve current Plan Version.
3. Create new Plan Version.
4. Copy unchanged assumptions.

5. Apply scenario-derived changes.
6. Create planned entity changes.
7. Create Adopted Decision record.
8. Store expected outcomes.
9. Link original Scenario.
10. Link SimulationRun.
11. Set new version as current.
12. Commit.

If any step fails:

ROLLBACK.

44. Version Strategy

Recommended version convention:

Minor operational decision:

1.0 → 1.1

Another operational decision:

1.1 → 1.2

Major strategic review:

1.2 → 2.0

Automated numbering must be atomic.

45. Conflict Detection

Example:

Current Live Plan:

Hire Employee A on 1 October

New scenario:

Cancel recruitment and use contractor

System should detect overlapping planned changes.

User should be shown:

This decision conflicts with an existing planned hiring decision.

Options:

Replace Existing Decision

Cancel New Adoption

The previous decision becomes:

SUPERSEDED

46. Actualisation of Planned Changes

A planned decision must eventually be compared against reality.

Example:

Plan:

Hire employee by 1 October.

On/after 1 October:

Check ontology:

Staff

If matching employee/role appears:

`planned_entity_change.status = COMPLETED`

If not:

`status = DUE`

After allowed grace period:

`status = MISSED`

47. Decision Performance Monitoring

Implement scheduled or event-driven checks.



Example:

Decision:

Increase Marketing Spend

Expected:

Revenue +15%

Customers +10/month

CAC $\leq$ £50

At target intervals:

Expected

vs

Actual

Store results in:

decision_outcome_observations

48. Decision Outcome Logic

Example:

variance $\leq$ tolerance

→ ON_TRACK

positive target achieved

→ ACHIEVED

material negative variance

→ UNDERPERFORMING

Repeated underperformance may update Adopted Decision to:

UNDERPERFORMING

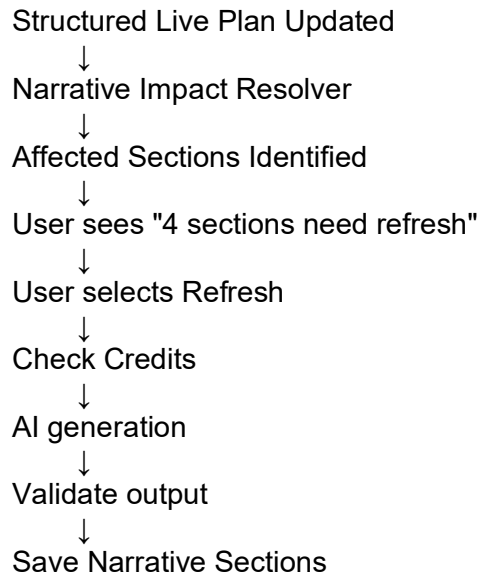
and generate:

CORRECTIVE_SCENARIO_RECOMMENDED

49. Narrative Architecture

Keep narrative generation separate from deterministic state management.

Sequence:



50. Narrative Impact Mapping

Example:

Pricing change affects:

Pricing Strategy
Financial Forecast
Revenue Model
Growth Strategy

Hiring affects:

Operational Plan
Staffing Strategy
Financial Forecast
Capacity Plan

Funding affects:

Funding Strategy
Cashflow
Growth Plan
Financial Strategy

51. Credit Integration

Reuse existing:

credit_wallets
credit_transactions
credit_feature_config
plan_credit_config

Do not create a separate Live Business Plan credit ledger.

52. Free Deterministic Actions

Credit cost:

0

for:

- KPI calculations
- variance calculations
- ontology synchronisation
- connector synchronisation
- deterministic assessments
- engine recalculations
- risk detection
- scenario calculations where no AI generation occurs
- scenario adoption
- plan version creation
- decision outcome monitoring
- plan-vs-actual calculations
- deterministic alerts
- structured planned-state updates

53. AI Credit Configuration

Recommended initial configuration:

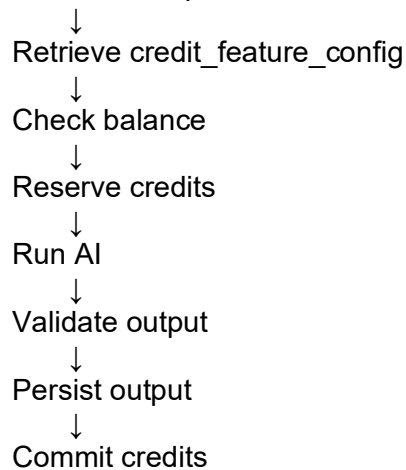
Feature Code	Feature	Credits
live_plan_import_extract	AI Business Plan extraction	10

Feature Code	Feature	Credits
live_plan_full_narrative	Generate full narrative	40
live_plan_section_refresh	Refresh one section	3
live_plan_variance_interpretation	AI variance interpretation	3
scenario_ai_interpretation	AI simulation interpretation	3
scenario_ai_recommendation	AI recommendation explanation	3
decision_ai_summary	AI decision summary	3
decision_narrative_update	Update narrative after adoption	5
live_plan_full_refresh	Full Live Plan narrative refresh	20
live_plan_monthly_review	AI monthly business review	10

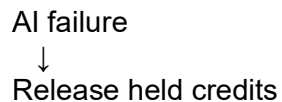
All values must remain admin-configurable.

54. Credit Execution Flow

AI Action Requested



Failure:



55. API Design

Live Plan

Create

POST /api/businesses/{businessId}/live-plan

Get

GET /api/businesses/{businessId}/live-plan

Adopt

POST /api/businesses/{businessId}/live-plan/adopt

Example:

```
{
  "source_type": "GENERATED_PLAN",
  "source_document_id": "uuid"
}
```

56. Plan Versions

GET /api/businesses/{businessId}/live-plan/versions

GET /api/businesses/{businessId}/live-plan/versions/{versionId}

GET /api/businesses/{businessId}/live-plan/versions/compare

Parameters:

version_a
version_b

57. KPIs

GET /api/businesses/{businessId}/live-plan/kpis

POST /api/businesses/{businessId}/live-plan/kpis

PATCH /api/businesses/{businessId}/live-plan/kpis/{kpId}

58. Plan vs Actual

GET /api/businesses/{businessId}/live-plan/performance

Example response:

```
{
  "period": "2026-10",
  "metrics": [
    {
      "metric_code": "revenue_monthly",
      "planned": 25000,
      "actual": 18000,
      "variance": -7000,
      "variance_pct": -28,
      "severity": "HIGH"
    }
  ]
}
```

59. Variances

GET /api/businesses/{businessId}/live-plan/variances

Filters:

period
severity
metric
domain
status

60. Alerts

GET /api/businesses/{businessId}/live-plan/alerts

POST /api/businesses/{businessId}/live-plan/alerts/{alertId}/acknowledge

POST /api/businesses/{businessId}/live-plan/alerts/{alertId}/dismiss

61. Scenario Adoption

Reuse existing scenario endpoints where practical.

Add:

POST /api/businesses/{businessId}/scenarios/{scenarioId}/adopt

Request:

```
{
  "simulation_run_id": "uuid",
  "effective_from": "2026-10-01",
  "decision_name": "Hire one operations employee"
}
```

62. Adoption Response

```
{
  "adopted_decision_id": "uuid",
  "previous_plan_version": "1.1",
  "new_plan_version": "1.2",
  "effective_from": "2026-10-01",
  "changed_assumptions": 6,
  "affected_narrative_sections": [
    "staffing_strategy",
    "operational_plan",
    "financial_forecast"
  ]
}
```

63. Decisions

GET /api/businesses/{businessId}/decisions

GET /api/businesses/{businessId}/decisions/{decisionId}

GET /api/businesses/{businessId}/decisions/{decisionId}/performance

64. Narrative Refresh

POST /api/businesses/{businessId}/live-plan/narrative/refresh

Request:

```
{
  "sections": [
    "staffing_strategy",
    "financial_strategy"
  ]
}
```

65. Required Services

Recommended domain services:

LivePlanService
PlanVersionService
PlanExtractionService
PlanValidationService
PlannedStateService
LivePlanKPIService
VarianceEngine
DependencyResolver
IntelligenceOrchestrator
ScenarioRecommendationService
DecisionAdoptionService
DecisionMonitoringService
NarrativeImpactResolver
LivePlanNarrativeService
LivePlanAlertService

Avoid putting business logic directly inside API controllers.

66. Event Model

Recommended domain events:

LIVE_PLAN_ADOPTED
LIVE_PLAN_VERSION_CREATED
BUSINESS_DATA_UPDATED
KPI_UPDATED
MATERIAL_VARIANCE_DETECTED
ASSESSMENT_UPDATED
RISK_FLAG_CREATED
SCENARIO_RECOMMENDED
SCENARIO_COMPLETED
DECISION_ADOPTED

PLANNED_CHANGE_DUE
PLANNED_CHANGE_COMPLETED
DECISION_UNDERPERFORMING
DECISION_ACHIEVED
NARRATIVE_REFRESH_REQUIRED

67. Transactional Outbox

For important events such as:

DECISION_ADOPTED
LIVE_PLAN_VERSION_CREATED

use a transactional outbox pattern if EnterprateAI already supports event processing.

Database changes and event creation should commit atomically.

This prevents:

Plan updated
but event lost

or:

Event dispatched
but database transaction failed.

68. Background Jobs

Recommended jobs:

KPI Refresh

Triggered by relevant business data updates.

Variance Evaluation

Runs after KPI calculation.

Assessment Refresh

Runs only affected engines.

Planned Change Monitor

Checks upcoming/due planned actions.

Decision Outcome Monitor

Evaluates adopted decision results.

Live Plan Health Check

Periodic integrity check.

69. Idempotency

All mutation endpoints must support idempotency.

Critical actions:

plan adoption
scenario adoption
narrative refresh
plan version creation
decision monitoring
integration-triggered KPI calculation

Never create duplicate Live Plan versions due to API retry.

70. Permissions

Roles:

OWNER / ADMIN

Can:

- adopt Live Plan
- change assumptions
- adopt decisions
- cancel decisions
- manage KPI targets
- refresh narratives

MANAGER

Can:

- view Live Plan

- run scenarios
- create proposed updates
- review risks

Scenario adoption configurable.

VIEWER / ADVISOR

Read-only.

71. Audit Requirements

Every material Live Plan mutation must record:

business_id
user_id
timestamp
operation
source
previous_value
new_value
plan_version
scenario_id
simulation_run_id
decision_id
reason

Audit records must be immutable.

72. Security

The Live Business Plan must respect existing:

- workspace isolation
- role-based access
- connector credential security
- encryption practices
- GDPR controls

Live Plan APIs must perform server-side:

user → workspace → business → entitlement

validation.

73. Subscription Entitlement

Before accessing Live Business Plan:

check subscription plan

Required:

Decision Engine
or higher

Starter users may see:

Live Business Plan
Upgrade to Decision Engine

but should not access the feature unless commercial rules later change.

74. Data Confidence

Create:

HIGH
MEDIUM
LOW

Recommended:

HIGH

Connected verified system data.

MEDIUM

Structured historical manual data.

LOW

Forecast assumptions / estimates.

Display confidence where intelligence relies heavily on estimates.

75. Engine Confidence

When running:

Viability
Survival
Stability
Growth

pass source-confidence metadata.

Example:

Runway = accounting integration
HIGH

versus:

Projected sales growth = user assumption
LOW

76. Performance Targets

Suggested application targets:

Standard KPI read:

< 500ms

Variance calculation:

< 500ms

Single deterministic simulation:

target < 1 second

Dashboard aggregate:

target < 2 seconds

AI narrative operations are asynchronous from the deterministic UI experience where appropriate.

77. Caching

Cache:

- current plan version
- active KPI configuration
- latest assessment
- current scores
- common dashboard aggregates

Do not cache:

- credit balances without correct invalidation
- mutation results
- security-sensitive entitlement decisions for excessive durations

78. Failure Handling

Integration failure:

Do NOT mark Live Plan as failed.

Instead:

Source: QuickBooks

Last successful sync: ...

Data freshness: STALE

79. Stale Data

Every source should maintain:

`last_synced_at`

If data exceeds configured freshness:

confidence downgrade

and display:

Financial data has not been updated recently.

Do not silently present stale data as current intelligence.

80. Partial Data

The system must work even if not all integrations are connected.

Example:

QuickBooks connected:

Financial confidence HIGH

No CRM:

Pipeline confidence LOW

EnterpriseAI must not require every possible data source.

81. Testing Requirements

Unit Tests

Required for:

- KPI calculations
- variance calculations
- planned-state overlay
- version numbering
- scenario adoption
- conflict detection
- decision outcome logic
- credit actions

82. Integration Tests

Test:

Connector data

→ Ontology

→ KPI

→ Variance

→ Engine

→ Alert

83. Scenario Adoption Test

Required end-to-end test:

Create Live Plan
↓
Run Hiring Scenario
↓
Verify actual Staff unchanged
↓
Adopt Hiring Scenario
↓
New plan version created
↓
Planned Employee stored
↓
Actual Staff still unchanged
↓
Effective date arrives
↓
Actual employee appears
↓
Planned change marked COMPLETED

This is a critical acceptance test.

84. Historical Integrity Test

Given:

Version 1.0 Revenue Target = £20k

Version 1.1 Revenue Target = £25k

Version 1.0 must always continue to return:

£20k

No later operation may mutate historical version data.

85. Credit Failure Test

Given:

AI narrative refresh = 5 credits

If AI provider fails:

Credits deducted = 0

Use existing reserve/commit/release logic.

86. Recommended Implementation Sequence

Sprint / Phase 1: Data Foundation

Build:

- live_business_plans
- live_plan_versions
- live_plan_assumptions
- planned_entity_changes
- plan_kpis
- kpi_observations
- plan_variances

Deliver:

Adopt Business Plan → structured Live Plan

87. Phase 2: Plan vs Actual

Build:

- ontology-to-KPI mapping
- KPI service
- integration data consumption
- variance engine
- plan performance API
- Live Plan dashboard

Deliver:

Plan vs Actual

88. Phase 3: Intelligence

Connect:

- Viability
- Survival
- Stability
- Growth
- Fragility/Master Score
- Risk flags
- dependency resolver

Deliver:

Variance → Risk Intelligence

89. Phase 4: Scenario Recommendation

Build:

- risk-to-scenario mapping
- recommended scenarios
- scenario launch from Live Plan

Deliver:

Detect → Simulate

90. Phase 5: Decision Adoption

Build:

- Adopted_decisions
- Expected outcomes
- DecisionAdoptionService
- planned-state overlay updates
- plan versioning
- conflict resolution

Deliver:

Simulate → Adopt

91. Phase 6: Decision Monitoring

Build:

- decision outcome observations
- decision monitoring jobs
- underperformance detection
- corrective scenario recommendation

Deliver:

Adopt → Monitor → Adapt

92. Phase 7: Narrative Intelligence

Build:

- affected-section detection
- AI section regeneration
- AI variance interpretation
- AI decision summary
- credit integration

Deliver:

Structured intelligence → readable business narrative

93. MVP Definition of Done

The MVP is considered technically complete when:

1. Existing/generated Business Plan can be adopted.
2. Live Plan Version 1.0 is created.
3. Plan assumptions are structured.
4. Plan assumptions map to ontology concepts.

5. Actual data can come from native EnterpriseAI or existing integrations.
6. KPI observations can be generated.
7. Plan-vs-actual variance can be calculated.
8. Material variance can trigger relevant engines.
9. Risk flags are generated.
10. Relevant scenarios can be recommended.
11. Scenario execution does not mutate Actual or Plan State.
12. Scenario results show cross-engine effects.
13. User can adopt a completed scenario.
14. Adoption creates a new immutable Live Plan version.
15. Adoption creates Planned-State changes.
16. Actual ontology records are not prematurely changed.
17. Expected decision outcomes are stored.
18. Adopted decisions can later be compared with actual outcomes.
19. AI narrative updates are optional and credit-controlled.
20. All major actions are auditable.

94. Critical Engineering Invariants

These rules must never be violated.

Invariant 1

Actual State is not Plan State.

A future plan must never become historical truth.

Invariant 2

Scenario State is not Plan State.

A scenario remains hypothetical until explicitly adopted.

Invariant 3

Plan versions are immutable.

Every approved change creates another version.

Invariant 4

Deterministic intelligence is the source of calculated truth.

AI explains the results.

Invariant 5

Scenario adoption is transactional.

Partially adopted decisions must never exist.

Invariant 6

Existing integrations feed the ontology.

Live Business Plan should not become another integration subsystem.

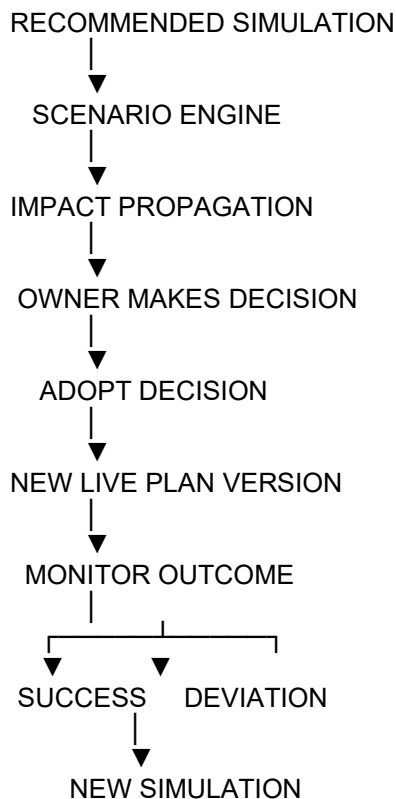
Invariant 7

Historical actuals must never be rewritten by Live Business Plan actions.

95. Strategic Architecture

The implementation should ultimately support this closed-loop system:





96. Final Engineering Requirement

The Live Business Plan must not be implemented as an automatically updating document.

It must be implemented as:

a version-controlled planned-state layer connected to EnterpriseAI's Business Ontology and Intelligence Engines.

The readable Business Plan is a presentation of that state.

The ontology represents actual business reality.

The Live Plan represents approved future intent.

The Scenario Engine represents hypothetical alternatives.

The Decision Adoption layer controls when a hypothetical alternative becomes an approved future plan.

The monitoring layer then determines whether reality ultimately matches the adopted decision.



This architecture gives EnterprateAI the technical foundation for:

Detect → Simulate → Decide → Adopt → Monitor → Adapt

and preserves the core innovation of modelling how one business decision propagates across the business before the owner commits to it.